

Preemptive scheduling mechanics and context switching overhead , Round-Robin (RR) scheduling and the impact of time quantum sizes , Priority-based scheduling and Shortest Remaining Time First (SRTF) , Multilevel Queue and Multilevel Feedback Queue scheduling

Code:

```
geya@LenovoL480: ~/OS/Pra  ×  +  v
GNU nano 8.7.1
#include <stdio.h>

int main() {

    int process[] = {1, 2, 3};
    int burst[] = {5, 3, 4};
    int priority[] = {2, 1, 3};
    int n = 3;

    printf("PREEMPTIVE CPU SCHEDULING\n\n");

    /* ----- ROUND ROBIN ----- */

    int remaining[3] = {5, 3, 4};
    int quantum = 2;
    int time = 0;

    printf("1. ROUND ROBIN\n");
    printf("Time Quantum = %d\n", quantum);

    while (remaining[0] > 0 ||
           remaining[1] > 0 ||
           remaining[2] > 0) {

        for (int i = 0; i < n; i++) {
```

```

    if (remaining[i] > 0) {
        printf("Time %d: P%d runs\n",
            time, process[i]);

        if (remaining[i] > quantum) {
            time += quantum;
            remaining[i] -= quantum;

            printf("Context switch after P%d\n",
                process[i]);
        }
        else {
            time += remaining[i];
            remaining[i] = 0;

            printf("P%d completed\n",
                process[i]);
        }
    }
}

```

```

/* ----- PRIORITY ----- */

printf("\n2. PRIORITY SCHEDULING\n");
printf("Lower number = Higher priority\n");

int completed[3] = {0, 0, 0};

for (int count = 0; count < n; count++) {
    int selected = -1;
    for (int i = 0; i < n; i++) {
        if (!completed[i]) {
            if (selected == -1 ||
                priority[i] < priority[selected]) {
                selected = i;
            }
        }
    }

    printf("P%d runs (Priority %d)\n",
           process[selected],
           priority[selected]);
}

```

```

        completed[selected] = 1;
    }

    /* ----- SRTF ----- */

    printf("\n3. SHORTEST REMAINING TIME FIRST\n");

    int srtf[3] = {5, 3, 4};
    int current_time = 0;

    while (srtf[0] > 0 ||
           srtf[1] > 0 ||
           srtf[2] > 0) {

        int shortest = -1;

        for (int i = 0; i < n; i++) {

            if (srtf[i] > 0) {

                if (shortest == -1 ||
                    srtf[i] < srtf[shortest]) {

                    shortest = i;
                }
            }
        }
    }

```

```

    }

    printf("Time %d: P%d runs\n",
           current_time,
           process[shortest]);

    srtf[shortest]--;
    current_time++;
}

/* ----- MULTILEVEL QUEUE ----- */

printf("\n4. MULTILEVEL QUEUE\n");

printf("Queue 1: System processes\n");
printf("Queue 2: User processes\n");

printf("Queue 1 -> P1\n");
printf("Queue 2 -> P2, P3\n");

printf("Higher priority queue executes first.\n");

/* ----- MULTILEVEL FEEDBACK QUEUE ----- */

printf("\n5. MULTILEVEL FEEDBACK QUEUE\n");

printf("P1 starts in Queue 1\n");
printf("If P1 uses too much CPU, move it to Queue 2\n");
printf("Short jobs remain in higher priority queue\n");

printf("\nScheduling demonstration completed.\n");

return 0;
}

```

Output:

```
geya@LenovoL480:~/OS/Practical/Week 7$ ./task1  
PREEMPTIVE CPU SCHEDULING
```

1. ROUND ROBIN

Time Quantum = 2

Time 0: P1 runs

Context switch after P1

Time 2: P2 runs

Context switch after P2

Time 4: P3 runs

Context switch after P3

Time 6: P1 runs

Context switch after P1

Time 8: P2 runs

P2 completed

Time 9: P3 runs

P3 completed

Time 11: P1 runs

P1 completed

2. PRIORITY SCHEDULING

Lower number = Higher priority

P2 runs (Priority 1)

P1 runs (Priority 2)

P3 runs (Priority 3)

3. SHORTEST REMAINING TIME FIRST

Time 0: P2 runs

Time 1: P2 runs

```
Time 9: P1 runs  
Time 10: P1 runs  
Time 11: P1 runs
```

4. MULTILEVEL QUEUE

Queue 1: System processes

Queue 2: User processes

Queue 1 -> P1

Queue 2 -> P2, P3

Higher priority queue executes first.

5. MULTILEVEL FEEDBACK QUEUE

P1 starts in Queue 1

If P1 uses too much CPU, move it to Queue 2

Short jobs remain in higher priority queue

Scheduling demonstration completed.

geya@LenovoL480:~/OS/Practical/Week 7\$ |